

Yellow Devil MOM

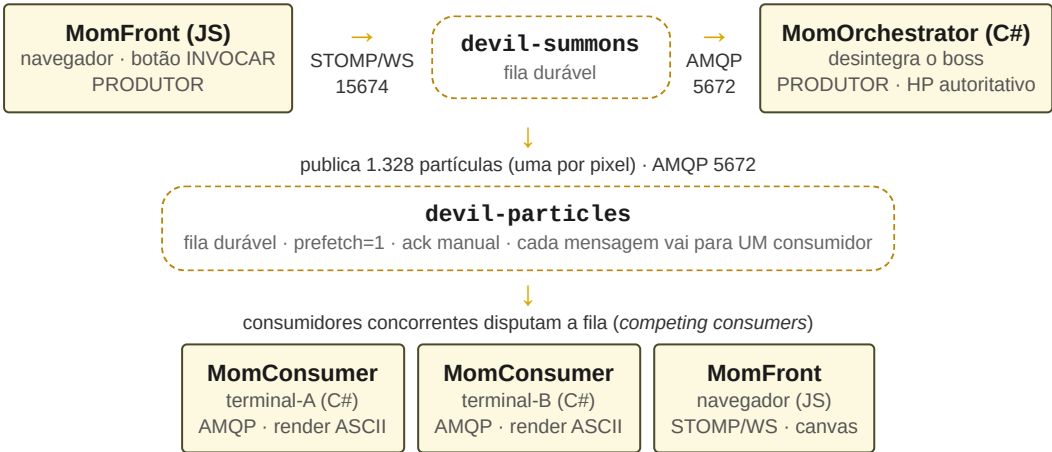
Teletransporte de um chefe de Mega Man (1987) via fila de mensagens (RabbitMQ)
Disciplina: Desenvolvimento de Sistemas Distribuídos | Ramon Couto Santos & Aaron Goldberg Guerra

Descrição da solução implementada

O Yellow Devil, chefe de *Mega Man* (1987), é famoso por **se desmontar em partículas**, atravessar a tela e **se remontar** do outro lado. Aqui essa mecânica vira um sistema distribuído: cada pixel não-vazio do sprite ASCII 52×36 — **1.328 partículas** — é **uma mensagem** publicada numa fila do RabbitMQ. Um **orquestrador em C#** desintegra o boss e publica as partículas embaralhadas em `devil-particles`; **consumidores concorrentes** disputam a fila e vão remontando o boss, cada um desenhando **apenas as partículas que pegou**: dois terminais C# com render ASCII e uma página web em JavaScript com `<canvas>` e barra de vida. Pelo navegador o usuário **invoca** o boss e **atira** nele, e essas ações também trafegam pelo broker (`devil-summons` e `devil-hits`) — logo o front é **produtor e consumidor ao mesmo tempo**. O back-end fala **AMQP** (5672) e o front fala **STOMP sobre WebSocket** (15674): duas linguagens e dois protocolos sobre o **mesmo broker** e as **mesmas filas**.

ABORDAGEM ESCOLHIDA
Opção A — Filas de Mensagens (comunicação ponto-a-ponto / *work queue*), com **RabbitMQ** como broker real: 1 produtor, filas duráveis e 3 consumidores concorrentes.

Diagrama da comunicação



O boss se divide entre os três: cada partícula é desenhada em um só lugar. Se um consumidor cai, as mensagens não confirmadas voltam à fila e são reentregues aos outros.

Justificativa da escolha

A regra do estudo de caso é que **cada partícula seja processada por exatamente um consumidor**: se duas janelas desenhassem o mesmo pixel, o boss apareceria *duplicado* em vez de *repartido*. Essa é a definição de **fila de trabalho ponto-a-ponto** — a fila entrega cada mensagem a um único consumidor e distribui a carga entre os que estiverem ativos. Com `prefetch=1` e **ack manual**, a divisão é justa e a falha fica visível: ao interromper um consumidor no meio do teleporte, as mensagens que ele não confirmou **voltam para a fila** e são **reentregues** a outro, sem perda. Em pub/sub cada assinante receberia uma cópia de *todas* as partículas e o boss seria montado inteiro em cada um — a divisão de trabalho, que é o coração da demonstração, desapareceria. Pub/sub serve para **notificar todos**; nós precisávamos **repartir tarefas**.

Capturas de tela do sistema em funcionamento

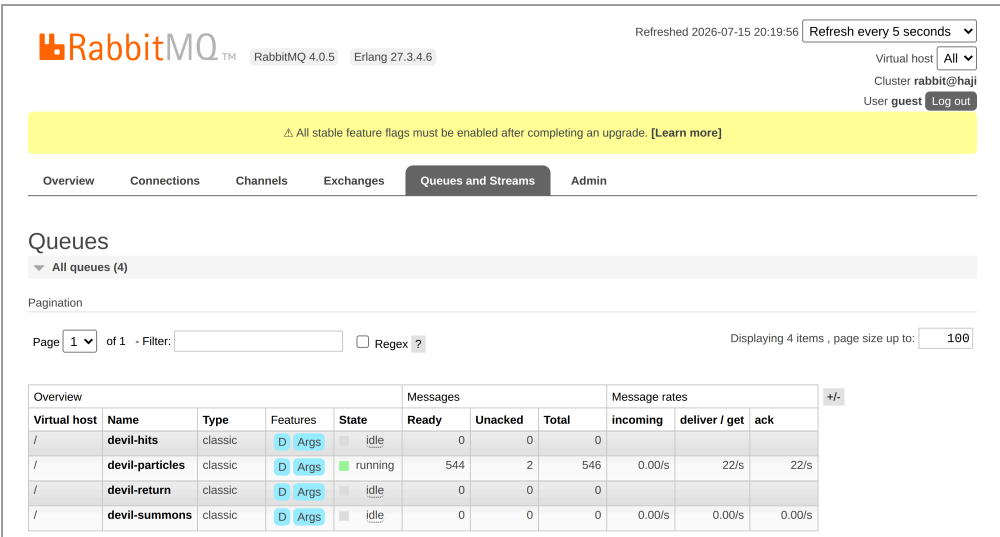


Figura 1 — Broker RabbitMQ 4.0.5 em execução (Management UI, localhost : 15672): as 4 filas duráveis (D) do estudo de caso. Durante um teleporte, devil-particles está running com 544 mensagens na fila, 2 unacked — o prefetch=1 de cada um dos 2 consumidores — e 22/s de entrega e de ack.

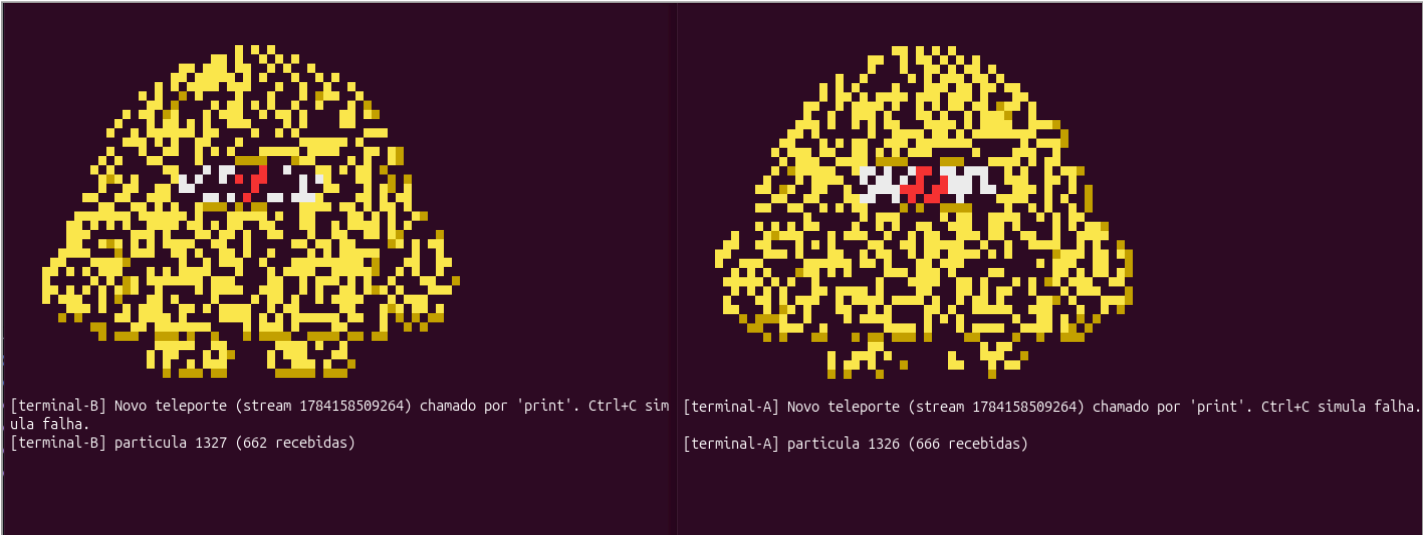


Figura 2 — Dois consumidores concorrentes disputando devil-particles no mesmo teleporte (stream 1784158509264): terminal-A (direita) recebeu 666 partículas e terminal-B (esquerda) recebeu 662 — 666 + 662 = 1.328, o total exato do sprite. Cada um desenha só a parte que pegou, por isso o boss aparece incompleto nos dois: nenhuma partícula foi processada duas vezes nem se perdeu.

Respostas objetivas

a) Qual paradigma foi implementado?

Middleware Orientado a Mensagens (MOM) com **filas ponto-a-ponto** (*work queue*) no RabbitMQ — Opção A. Um produtor (o orquestrador C#) publica em filas duráveis e consumidores concorrentes (dois terminais C# via AMQP e o front JavaScript via STOMP/WebSocket) disputam a fila devil-particles. Comunicação assíncrona, com prefetch=1 e ack manual.

b) Por que esse paradigma é adequado ao estudo de caso escolhido?

Porque cada partícula precisa ser tratada **uma única vez** e o trabalho deve ser **repartido** entre os consumidores. A fila faz exatamente isso: entrega cada mensagem a um só consumidor, balanceia a carga entre os ativos e, com ack manual, **reentrega** o que um consumidor deixou de confirmar ao falhar. Também desacopla no tempo — o orquestrador publica mesmo sem nenhum consumidor online.

c) Quais seriam as principais mudanças caso fosse utilizada a outra abordagem?

Trocaríamos a fila direta por uma **exchange fanout**, e cada assinante teria a **sua própria fila** ligada a ela. Consequência: **todos** receberiam **todas** as partículas — o boss seria montado inteiro em cada assinante, em vez de dividido. Deixaria de existir "processado exatamente uma vez" e divisão de carga; viraria um modelo de broadcast/notificação.

d) Como o broker contribui para o desacoplamento entre os componentes da aplicação?

O RabbitMQ fica **no meio**: produtores e consumidores só conhecem o **nome da fila**, nunca uns aos outros. Isso desacopla **no tempo** (filas duráveis guardam as mensagens até alguém consumir), **no espaço** (ninguém precisa saber endereço ou porta de ninguém, só do broker) e **em linguagem e protocolo**: o C# fala AMQP e o navegador fala STOMP/WebSocket sobre as **mesmas** filas.